

PyCAPGE: A Python Engine for Classic Adventure Game Education

Eduardo Garbayo ¹

¹ Zainder.com, Spain

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Submitted: 11 January 2026

Published: unpublished

License

Authors of papers retain copyright¹ and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/))²

Summary

PyCAPGE (Python Classic Adventure Point-and-Click Game Engine) is an open-source engine built upon the Pygame library (providing an abstraction over SDL2) for building classic-style graphical adventure games in Python. It reimplements point-and-click adventure mechanics (navigable rooms with hotspots, inventory, dialogs, and puzzles) in a modern, data-driven architecture. All core systems (main loop, finite-state machine, resource management) are visible in the code rather than hidden behind an editor. For example, scene transitions and item interactions are scripted in Python (often via simple callback tables rather than a graphical GUI). This preserves the logic of classic games while making the internals explicit, so that students can inspect how scenes, objects, and interactions are implemented.

Statement of need

Teaching object-oriented programming often suffers from an “abstraction gap” (Kolling, 1999). Basic tools (Scratch, Pygame Zero) simplify syntax but hide architecture, while professional engines (Unity, Godot) are so complex that they obscure core concepts. PyCAPGE fills this gap by providing a complete adventure game engine that remains small and readable. It implements the discrete gameplay of point-and-click adventures (modeled as finite-state machines (Sicart, 2008)) using straightforward object-oriented code. For instance, a Hotspot class represents any interactive object and a Scene class represents a location. Abstract concepts become concrete: an AnimatedHotspot extends Hotspot with animation (illustrating inheritance), and verb commands map directly to callback functions. Each line of code produces visible results in the running game, making learning engaging.

Beyond pedagogy, PyCAPGE serves a critical role in digital preservation and software archaeology. By reimplementing the logic of 1990s engines in readable high-level code—rather than simply emulating binaries—it provides an accessible artifact for researchers studying the history of game development mechanics and engine architecture.

The design of PyCAPGE is informed by the author’s two decades of experience in engine development, including a C-based implementation in 1997 and a Java-based adventure system in 2001. Lessons learned from the complexity of those earlier iterations directly influenced the choice of Python and the simplified architecture presented here.

To support these educational needs, PyCAPGE is released under GPLv3 on GitHub and SourceForge (Garbayo, n.d.-c, n.d.-b) with complete source code, documentation, and a playable demo archived on Zenodo (Garbayo, n.d.-a). Its lightweight, well-documented design facilitates use in both education and research; instructors can run the demo scenes to illustrate concepts like event loops, collision detection, or inventory management with live

41 examples. The modular codebase invites extensions (e.g. adding new verbs or localization)
 42 and signals community readiness for contributions. We anticipate that PyCAPGE will
 43 be adopted by educators and researchers interested in computing pedagogy, game-based
 44 learning, and digital preservation.

45 State of the field

46 Existing tools for game-based learning lie at two extremes. Scratch or basic Pygame
 47 tutorials (Sweigart, 2012) let novices animate sprites but provide no engine structure
 48 (no scene manager, no built-in item systems). Conversely, Unity and Godot (Unity
 49 Technologies, 2023) offer full game frameworks, but their rich editors and autogenerated
 50 code introduce high overhead and obscure the program's flow from beginners. Adventure-
 51 specific tools like Adventure Game Studio (AGS) let designers build games with visual
 52 editors and proprietary scripts, but they do not reveal implementation details. Likewise,
 53 systems such as ScummVM (ScummVM Team, 2001) run classic titles on modern hardware,
 54 but their C++ codebases are meant for playing games, not for teaching. PyCAPGE
 55 differentiates itself by occupying the middle ground: it is implemented in standard Python
 56 (using SDL2/Pygame) under a GPLv3 license, and all game logic is defined in readable
 57 code. This transparency makes the link between design and implementation explicit and
 58 accessible to learners.

59 Software design

60 PyCAPGE's architecture is arranged in clear layers for education. Global configuration
 61 (Python modules and YAML files) defines constants (screen size, speeds) and data (scene
 62 identifiers, text, inventory items). The core engine layer (classes.py) provides fundamental
 63 classes like Scene, Hotspot, and Character, and implements systems such as collision
 64 detection (via pixel-masks) and pathfinding (grid-based A*). To support comparative
 65 analysis, the engine exposes configurable algorithms (e.g., allowing students to toggle
 66 between Euclidean and Manhattan distance heuristics) and variable text visualization styles
 67 (simulating classic LucasArts floating text versus Sierra portrait overlays). Furthermore, to
 68 optimize performance on educational hardware, the engine implements a ResourceManager
 69 with lazy loading and caching, ensuring assets are loaded into memory only when required
 70 by the active scene. A SceneManager loads scene definitions; the main program (main.py)
 71 initializes subsystems (audio, input, resource manager) and enters the main loop (Gregory,
 72 2018). This loop is a finite-state machine: in an "Explore" state it handles character
 73 movement and interactions, whereas in a "Dialogue" state it blocks movement and advances
 74 conversation. This explicit state machine design teaches how to coordinate update and
 75 render steps across modes (Gamma et al., 1994).

76 Gameplay logic is defined declaratively in Python. Each interactive object is created in
 77 code with its position, sprite, and a dictionary of actions. Keys are verb-object strings
 78 (e.g. "USE_KEY_ON_DOOR") and values are Python lambda callbacks. When executed, these
 79 callbacks update the GAME_STATE dictionary, play sounds, or change inventory. This event-
 80 driven model demonstrates that functions are first-class: behaviors can be passed around
 81 and invoked at runtime. Students populate scenes by instantiating objects in code rather
 82 than using a visual editor, reinforcing the distinction between classes (templates) and
 83 instances. The game state (inventory, puzzle flags, etc.) is stored in Python dictionaries
 84 that can be saved to disk (JSON), illustrating persistence.

85 **AI usage disclosure**

86 Generative AI tools were used to assist the author during the development of this project.
87 These tools supported tasks such as translating the manuscript into English, improving
88 clarity and academic style, debugging code, and answering programming-related questions
89 during development. All software design decisions, final implementations, and scientific
90 claims were made by the author, who takes full responsibility for the correctness, originality,
91 and integrity of both the software and the manuscript.

92 Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of*
93 *reusable Object-Oriented software*. Addison-Wesley.

94 Garbayo, E. (n.d.-a). *PyCAPGE (software)*. <https://doi.org/10.5281/zenodo.18183594>.
95 <https://doi.org/10.5281/zenodo.18183594>

96 Garbayo, E. (n.d.-b). *PyCAPGE (SourceForge project)*. [https://sourceforge.net/projects/](https://sourceforge.net/projects/pycapge/)
97 [pycapge/](https://sourceforge.net/projects/pycapge/).

98 Garbayo, E. (n.d.-c). *PyCAPGE: Python classic adventure Point-and-Click game engine*.
99 <https://github.com/tijuinem/pycapge>.

100 Gregory, J. (2018). *Game engine architecture* (3rd ed.). CRC Press.

101 Kolling, M. (1999). The problem of teaching Object-Oriented programming, part 1:
102 languages. *Journal of Object-Oriented Programming*, 11(8), 8–15.

103 ScummVM Team. (2001). *ScummVM: Script creation utility for maniac mansion virtual*
104 *machine*. <https://www.scummvm.org/>.

105 Sicart, M. A. (2008). Defining game mechanics. *Game Studies*, 8(2).

106 Sweigart, A. (2012). *Making games with Python & Pygame*. CreateSpace Independent
107 Publishing Platform.

108 Unity Technologies. (2023). *Unity user manual*. [https://docs.unity3d.com/Manual/index.](https://docs.unity3d.com/Manual/index.html)
109 [html](https://docs.unity3d.com/Manual/index.html)